



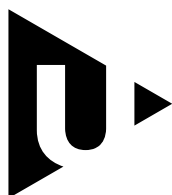
FAKULTA APLIKOVANÝCH VĚD
ZÁPADOČESKÉ UNIVERZITY
V PLZNI

KATEDRA INFORMATIKY
A VÝPOČETNÍ TECHNIKY

Bakalářský projekt

Hybridní generativní algoritmus spojující Wave Function Collapse s Ant Colony Optimization

Tomáš Rybák



FAKULTA APLIKOVANÝCH VĚD
ZÁPADOČESKÉ UNIVERZITY
V PLZNI

KATEDRA INFORMATIKY
A VÝPOČETNÍ TECHNIKY

Bakalářský projekt

Hybridní generativní algoritmus spojující Wave Function Collapse s Ant Colony Optimization

Tomáš Rybák

© Tomáš Rybák, 2026

Všechna práva vyhrazena. Žádná část tohoto dokumentu nesmí být reprodukována ani rozšiřována jakoukoli formou, elektronicky či mechanicky, fotokopírováním, nahráváním nebo jiným způsobem, nebo uložena v systému pro ukládání a vyhledávání informací bez písemného souhlasu držitelů autorských práv.

Citace v seznamu literatury:

RYBÁK, Tomáš. *Hybridní generativní algoritmus spojující Wave Function Collapse s Ant Colony Optimization*. Plzeň, 2026. Bakalářský projekt. Západočeská univerzita v Plzni, Fakulta aplikovaných věd, Katedra informatiky a výpočetní techniky.

Abstrakt

Tato práce se zabývá problematikou procedurálního generování komplexních 3D a 2D herních prostředí. Tradiční algoritmus Wave Function Collapse (WFC), ačkoliv dosahuje vysoké vizuální kvality na lokální úrovni, často selhává při generování globálních struktur (např. průchozích cest) a na větších mapách podléhá kombinatorické explozi. Cílem práce je návrh a implementace hybridního generátoru v prostředí Unity, který tyto nedostatky eliminuje. Řešení integruje horolezecký algoritmus (Hill Climbing) a optimalizaci mravenčí kolonie (ACO) jako předpočítací vrstvu konstruuující garantovanou průchozí páteř, na kterou WFC kolabuje okolní prostředí. Původně uvažovaná integrace formou měkkých omezení byla vyhodnocena jako architektonicky riziková a zůstává otevřenou otázkou pro navazující práci. Výsledky testování prokázaly, že hybridní model využívající ACO si zachovává stabilitu a generuje plně průchozí prostředí, zatímco čisté WFC a horolezecký crawler na rozlehlých mapách naráží na neřešitelné kontradikce.

Abstract

This thesis deals with the procedural generation of complex 3D and 2D game environments. The traditional Wave Function Collapse (WFC) algorithm, while achieving high visual quality on a local level, often fails to generate global structures (e.g., guaranteed paths) and suffers from combinatorial explosion on larger maps. The aim of this work is to design and implement a hybrid generator in the Unity engine that eliminates these shortcomings. The solution integrates a Hill Climbing crawler and Ant Colony Optimization (ACO) as a pre-calculation layer constructing a guaranteed traversable spine, onto which WFC collapses the surrounding environment. The originally considered soft constraint integration was evaluated as architecturally risky and remains open for future work. Testing results demonstrated that the hybrid ACO model maintains stability and generates fully traversable environments, whereas pure WFC and the Hill Climbing crawler encounter unresolvable contradictions on large maps.

Klíčová slova

procedurální generování • wave function collapse • ant colony optimization

Poděkování

Rád bych na tomto místě poděkoval svému vedoucímu práce, Mgr. Martinu Maňákoví, Ph.D., za odborné vedení, cenné rady a čas, který mi během zpracování tohoto projektu věnoval. Jeho rady významně přispěly k výsledné podobě této práce a k mému osobnímu růstu.

Obsah

Prohlášení o použití nástrojů umělé inteligence	4
Zadání projektu	5
1 Úvod	6
1.1 Kontext a motivace	6
1.2 Definice problému	6
1.3 Cíle práce	7
2 Teoretická východiska	8
2.1 Wave Function Collapse (WFC)	8
2.1.1 Princip superpozice a diskrétní prostor	8
2.1.2 Entropie a výběr buňky (Observation)	9
2.1.3 Propagace pravidel (Constraint Propagation)	9
2.1.4 Problém kontradikce a kombinatorická exploze	10
2.2 Prohledávání stavového prostoru	10
2.2.1 Prohledávání do hloubky (DFS)	10
2.2.2 Heuristické vyhledávání a Best-First Search	11
2.2.3 Lokální vyhledávání a horolezecký algoritmus (Hill Climbing)	11
2.3 Ant Colony Optimization (ACO)	12
2.3.1 Biologická inspirace a feromonová komunikace	12
2.3.2 Princip algoritmu a umělí agenti	12
2.3.3 Aktualizace stop: Odpařování a posilování	13
2.3.4 Aplikace v generování herních prostředí (Integrace s WFC)	13
3 Architektura systému	14
3.1 Reprezentace prostoru	14
3.2 Sockety a rotace modulů	14
3.3 Generativní solvery	15
3.3.1 WFCSolver	15
3.3.2 GLSHybridSolver	16

3.3.3	ACOHybridSolver	16
4	Implementace algoritmů	17
4.1	Wave Function Collapse	17
4.2	Greedy Local Search	17
4.2.1	Praktické nasazení v externím projektu	18
4.3	Ant Colony Optimization	19
5	Řídící logika a uživatelské rozhraní	21
5.1	Řídící logika	21
5.2	Uživatelské rozhraní a ochrana proti chybným vstupům	21
5.3	Režim izolované trasy	23
5.4	Minimapa	24
6	Testování, výsledky a diskuze	25
6.1	Metodika testování	25
6.2	Optimalizace parametrů ACO	25
6.2.1	Fáze 1 — hrubé prohledávání	26
6.2.2	Fáze 2 — jemné prohledávání	26
6.2.3	Fáze 3 — verifikace	27
6.3	Výkonnostní srovnání výběru entropie ve WFC	28
6.4	Srovnání solverů GLS a ACO	29
6.5	Finální verifikace ACO se sqrt-based škálováním	33
6.6	Chování na řádově větších mřížkách	34
6.6.1	Sqrt-based škálování na velkých konfiguracích	34
6.6.2	Manuální parametry	35
6.7	Vizuální analýza vlivu parametru beta	36
6.8	Diskuze a limitace	38
6.8.1	Odchylka od zadání: hard constraint namísto soft constraint	38
6.8.2	Reprodukovatelnost výsledků	38
6.8.3	Přenositelnost optimálního beta	39
6.8.4	Absence automatizované verifikace výstupu	39
6.8.5	Škálovatelnost nad 4 500 buněk	39
7	Závěr	40
A	Uživatelská příručka	42
A.1	Spuštění a hlavní menu	42
A.2	Generování prostředí	43
A.3	Pohyb ve vygenerovaném prostředí	43
A.4	Minimapa a diagnostické nástroje	43

Bibliografie

45

Prohlášení o použití nástrojů umělé inteligence

Během přípravy této práce autor použil následující nástroje umělé inteligence:

- **Gemini Pro 3.1 (Google)** – ke konzultacím týkajícím se implementačních detailů a návrhu softwarové architektury generativního systému.
- **Claude Sonnet 4.6 (Anthropic)** – k asistenci při formulaci a stylistické úpravě odborného textu a ke kontrole terminologické konzistence.

Po použití těchto nástrojů autor pečlivě zkontroloval a upravil veškerý obsah podle potřeby. Veškerá rozhodnutí týkající se návrhu, implementace a interpretace výsledků jsou vlastní prací autora. Autor přebírá plnou odpovědnost za výsledný obsah práce.

Zadání projektu

Bakalářský projekt v programu, specializaci Informatika – počítačové vědy.

Název: Hybridní generativní algoritmus spojující Wave Function Collapse s Ant Colony Optimization

Popis tématu:

Wave Function Collapse (WFC) je algoritmus procedurálního generování prostředí založený na aplikaci lokálních pravidel a respektující omezující podmínky. Ant Colony Optimization (ACO) je přístup inspirovaný chováním mravenců pro řešení úloh typu nalezení minimální cesty v dynamickém prostředí, aproximace řešení problému obchodního cestujícího a jiných. Cílem projektu je prozkoumat možnosti generování prostředí pomocí WFC, ve kterém se bude využívat zpětná vazba od ACO pro ovlivnění pravděpodobnosti aplikace lokálních pravidel.

Zadané úkoly:

1. Seznamte se s teoretickými podklady WFC a ACO.
2. Navrhněte a implementujte hybridní algoritmus kombinující WFC se zpětnou vazbou od ACO.
3. Vytvořte a proveďte experimenty zkoumající chování navrženého hybridního přístupu.
4. Kriticky zhodnoťte dosažené výsledky.

Téma vypsál: Mgr. Martin Maňák, Ph.D. (UN 356)

Vypsáno pro akademický rok: 2025/2026

Datum vypsání: 23. ledna 2026

Zadáno: Tomáš Rybák

1.1 Kontext a motivace

V současném herním průmyslu a v oblasti simulací virtuálních prostředí neustále roste poptávka po rozsáhlém a unikátním obsahu. Tradiční manuální tvorba herních světů je procesem časově i finančně nákladným, což vede k rozmachu metod procedurálního generování obsahu (PCG – *Procedural Content Generation*). Cílem těchto technik je automatizovat tvorbu komplexních struktur při zachování vysoké míry variability a estetické kvality. Přechod od 2D generování k plně trojrozměrným (3D) strukturám navíc přináší nezanedbatelný nárůst složitosti. Zajistit smysluplnou vertikální a topologickou návaznost v 3D prostoru tak, aby výsledné prostředí (například víceúrovňový dungeon) působilo přirozeně a bylo plně hratelné, představuje jednu z výzev současného procedurálního designu.

Spektrum PCG metod je široké a volba přístupu závisí na charakteru generovaného obsahu. Pro generování terénu a přírodních krajin se uplatňují šumové funkce (*noise functions*), zejména Perlinův šum [STN16]. Pravidlová gramatika (*L-systems*, *shape grammars*) nachází uplatnění při generování vegetace a architektonických struktur. Pro generování bludišť a uzavřených prostorů se tradičně využívají metody BSP stromů (*Binary Space Partitioning*) nebo celulární automaty. Všechny tyto přístupy optimalizují různé vlastnosti výstupu – vizuální věrnost, výpočetní efektivitu nebo kontrolovatelnost struktury – a žádný z nich neposkytuje univerzální řešení. Algoritmus Wave Function Collapse, představený v následující sekci, spadá do kategorie metod založených na omezujících podmínkách (*constraint-based methods*) a vyniká zejména schopností zachovat lokální strukturální vzory předlohy.

1.2 Definice problému

Wave Function Collapse (WFC) je jedním z algoritmů, který v posledních letech nachází uplatnění v procedurálním generování obsahu. Tento přístup, inspirovaný principy kvantové mechaniky a diskrétními modely, dokáže generovat logicky spojené prostředí na základě lokálních syntaktických pravidel a omezení. Ačkoliv

WFC vyniká v udržování lokální konzistence a vizuální věrnosti (velmi dobře funguje pro generování textur či spojování lokálních vzorů), vykazuje zásadní slabinu v absenci globálního kontextu. Algoritmus primárně nedisponuje mechanismem pro zajištění komplexních vlastností, jako je například garantované propojení startu a cíle v rozlehlém prostoru.

V případech, kdy lokální pravidla vedou k uzavření cesty dříve, než je dosaženo cíle, dochází k tzv. kontradikci. Ta vyžaduje nákladné restartování celého generativního procesu, což činí čisté WFC v praxi neefektivním a nestabilním nástrojem pro generování garantovaně průchozích úrovní. Tento jev byl empiricky doložen i v příbuzné oblasti generování bludišť na grafových strukturách – bez mechanismu garantujícího průchoznost dosahují stochastické generátory míry řešitelnosti výsledné mapy pohybující se v rozsahu 8-50 % v závislosti na kombinaci pravidel [Zap23, Tab. 4.2].

1.3 Cíle práce

Cílem této práce je návrh a implementace hybridního generátoru, který tyto nedostatky řeší. Koncept kombinuje lokální strukturální pravidla WFC s algoritmy lokálního vyhledávání (*Hill Climbing*) a globální optimalizací založenou na mravenci kolonii (ACO – *Ant Colony Optimization*).

Heuristika ACO, založená na principech rojové inteligence a chování mravenců při hledání potravy, je navržena pro řešení optimalizačních úloh v grafových strukturách. Při návrhu implementace byly zvažovány dva způsoby integrace ACO s WFC. První přístup, odpovídající zadání práce, využívá *měkkých omezení* (*soft constraints*): feromonové hodnoty jsou integrovány přímo do kolapsové funkce jako váhy při výběru varianty z S_c , čímž ACO statisticky ovlivňuje lokální rozhodování WFC. Tento přístup byl vyhodnocen jako architektonicky rizikový — nesprávně nastavený feromonový gradient může frekvenci kontradikčních stavů zvýšit, nikoliv snížit, protože preferovaná varianta nemusí být lokálně kompatibilní s již zkolabovanými sousedy. Implementace proto využívá *pevná omezení* (*hard constraints*): ACO vrstva předpočítá garantovanou průchozí trasu jako strukturální předpoklad, na který je následně aplikován WFCSolver. Vliv ACO na výsledné prostředí je tak deterministický na úrovni globální topologie, nikoliv probabilistický na úrovni lokálních voleb.

Teoretická východiska

2

2.1 Wave Function Collapse (WFC)

Algoritmus *Wave Function Collapse* (WFC) představil v roce 2016 softwarový vývojář Maxim Gumin [Gum16] původně jako nástroj pro procedurální generování dvourozměrných bitmap na základě lokální podobnosti se vstupním vzorem. Název algoritmu je volně inspirován kvantovou mechanikou, přičemž stav před vygenerováním konečného výsledku je chápán jako kvantová superpozice všech možných stavů.

Následný akademický výzkum, prezentovaný v práci Kartha a Smithe [KS17], formálně zařadil WFC do rodiny algoritmů pro řešení omezujících podmínek (CSP – *Constraint Satisfaction Problem*). V kontextu trojrozměrného generování herních prostředí se algoritmus typicky nespolehá na analýzu překrývajících se pixelů, ale využívá tzv. *Simple Tiled Model* (jednoduchý dlaždicový model). Tento model operuje s předem definovanými stavebními bloky a pravidly jejich propojování. Úspěšnou aplikaci tohoto přístupu pro generování 3D měst představil Marian Kleineberg [Kle19]. Architektura předkládané práce na tyto principy navazuje a implementuje je formou diskrétní trojrozměrné mřížky.

2.1.1 Princip superpozice a diskrétní prostor

Základním stavebním prvkem implementovaného systému je logická buňka (v architektuře reprezentována třídou `Cell`). Na počátku generování se každá buňka v mřížce nachází v plné superpozici. To znamená, že místo jednoho konkrétního stavebního dílku obsahuje seznam všech teoreticky možných variant (tzv. `AvailableVariants`), které na danou souřadnici lze umístit.

Každý stavební dílek (`TileVariant`) má v rámci systému přesně definované rozhraní pomocí šesti směrových socketů (Sever, Východ, Jih, Západ, Nahoru, Dolů). Proces generování mapy v pojetí WFC tedy nespočívá v postupném přidávání bloků

do prázdného prostoru, ale v odstraňování (redukci) nekompatibilních možností z původní superpozice na základě sousedských pravidel.

2.1.2 Entropie a výběr buňky (Observation)

Proces postupného určování konečného stavu buněk se v terminologii algoritmu označuje jako pozorování (*Observation*). V každém kroku algoritmus vybírá jednu buňku, kterou zkolabuje do jediného, finálního stavu.

Výběr této buňky není náhodný, ale řídí se heuristikou minimální entropie. V kontextu této implementace představuje entropie míru nejistoty, která je matematicky zjednodušena na prostý počet dostupných variant, což se liší od originální implementace, která používá Shannonovu entropii. Algoritmus z mřížky vždy vybírá tu buňku, jejíž entropie je nejnižší (větší než 1). Tento deterministický přístup zajišťuje, že systém řeší nejprve ta místa v prostoru, která jsou nejvíce prostorově omezena svými sousedy, čímž se minimalizuje riziko chybné volby.

2.1.3 Propagace pravidel (Constraint Propagation)

Základním principem algoritmu zajišťujícím lokální konzistenci mřížky je šíření omezení (*Constraint Propagation*). Jakmile dojde ke kolapsu vybrané buňky do jediného stavu, její superpozice se redukuje na jedinou variantu a tato změna musí být propagována do okolí.

Šíření omezení je implementováno iterativně pomocí zásobníku (Stack). Po kolapsu buňky jsou do zásobníku vloženi její sousedé, jejichž množiny přípustných variant je třeba přezkoumat. Z každého souseda jsou odebrány varianty nekompatibilní se socketem zkolabované buňky v příslušném směru; pokud dojde ke změně, je soused vložen do zásobníku a proces pokračuje iterativně. Pořadí zpracování dané zásobníkem (LIFO) nemá vliv na výslednou množinu přípustných variant – použití fronty (BFS) by vedlo ke stejné výsledné konfiguraci hranové konzistence, avšak s potenciálně odlišným počtem opakovaných zpracování téže buňky.

V terminologii teorie omezení tento proces odpovídá dosažení tzv. hranové konzistence (*arc consistency*): pro každou dvojici sousedních buněk (c_i, c_j) a každou variantu $v \in S_{c_i}$ existuje alespoň jedna varianta $v' \in S_{c_j}$, jejíž socket v příslušném protilehlém směru je s v kompatibilní [KS17]. Hranová konzistence zaručuje lokální kompatibilitu sousedních párů buněk; nezaručuje však existenci průchozí cesty jako globální vlastnosti prostoru – tato limitace je primární motivací pro hybridní vrstvu popsanou v následujících kapitolách.

2.1.4 Problém kontradikce a kombinatorická exploze

Ačkoliv heuristika minimální entropie a propagace pravidel činí algoritmus velmi silným v generování lokálně soudržných struktur, naráží WFC na podstatné limity algoritmů typu CSP.

Stavový prostor 3D mřížky o N buňkách, z nichž každá může nabývat jedné z k variant, má kardinalitu k^N . Nalezení přiřazení splňujícího všechna omezení odpovídá prohledávání tohoto prostoru, jehož velikost roste exponenciálně s rozměrem mřížky a počtem vstupních variant – tento jev se označuje jako kombinatorická exploze (*combinatorial explosion*) [RN21].

Nejvýraznějším technickým projevem těchto limitů je **kontradikce**. Ta nastává, když se při řetězové propagaci omezení vyprázdní množina přípustných variant některé buňky $S_c = \emptyset$. Algoritmus se ocitne ve stavu, z něhož nelze pokračovat bez porušení platných omezení. Standardní WFC v této architektuře nevyužívá *backtracking*, neboť odvolávání hluboce zanořených řetězových propagací je výpočetně neúnosné. Výskyt kontradikce proto vede k selhání generátoru a nutnosti resetovat celou mřížku. Právě tento strukturální nedostatek, jenž zabraňuje garantování existence průchozí cesty ve složitém prostoru, je primární motivací pro integraci hybridní vrstvy popsané v následujících kapitolách.

2.2 Prohledávání stavového prostoru

Hybridní přístup navržený v této práci předpočítává páteřní trasu před spuštěním algoritmu WFC. Cílem prohledávání je nalezení posloupnosti buněk $\langle b_0, b_1, \dots, b_n \rangle$ spojující počáteční buňku b_0 s cílovou buňkou b_n , přičemž každý přechod (b_i, b_{i+1}) musí splňovat kompatibilitu socketů sousedních buněk. Následující sekce popisují algoritmy prohledávání stavového prostoru, jejichž vlastnosti a limity motivují volbu konkrétní strategie použité v implementaci.

2.2.1 Prohledávání do hloubky (DFS)

Z pohledu teorie grafů lze diskretní mřížku buněk chápat jako neorientovaný graf, jehož uzly jsou jednotlivé buňky a hrany reprezentují sousedské vztahy. Prohledávání do hloubky (*Depth-First Search*, DFS) je jedním ze základních algoritmů pro systematické procházení takového grafu [Cor+22].

Princip algoritmu spočívá ve sledování jediné větve grafu tak hluboko, jak je to jen možné. K implementaci slouží datová struktura zásobníku (*LIFO*). Jakmile algoritmus narazí na uzel bez nenavštívených hran (slepá ulička), provede krok zpět (*backtracking*) a pokračuje jinou větví z předchozího uzlu. Z hlediska teorie umělé inteligence se jedná o neinformovanou (*uninformed*) strategii. Ačkoliv disponuje

garantovanou úplností (řešení vždy nalezne, pokud existuje), absence heuristiky směřující hledání k cíli vede k neefektivnímu prozkoumávání slepých větví [RN21].

2.2.2 Heuristické vyhledávání a Best-First Search

Popsaný nedostatek motivuje přechod k informovaným (*informed*) strategiím. Tyto algoritmy při rozhodování využívají heuristickou funkci $h(n)$, která odhaduje vzdálenost aktuálního uzlu n od cílového uzlu g . V prostorovém prohledávání na diskretních mřížkách, kde je pohyb omezen na ortogonální směry bez možnosti diagonálního přesunu, je přirozenou volbou Manhattanská vzdálenost. V trojrozměrném prostoru je definována jako součet absolutních diferencí jednotlivých souřadnic:

$$h(n) = |x_n - x_g| + |y_n - y_g| + |z_n - z_g| \quad (2.1)$$

Tato metrika je pro daný typ mřížky přípustná (*admissible*), neboť nepřeceňuje skutečný minimální počet kroků k cíli. V ortogonální mřížce bez diagonálních přechodů odpovídá manhattanská vzdálenost přesně minimálnímu počtu kroků mezi dvěma uzly, čímž představuje těsný dolní odhad skutečné ceny cesty. Euklidovská vzdálenost by naopak skutečnou navigační cenu podceňovala – je sice také přípustná, avšak méně těsná, neboť předpokládá přímočarý pohyb, který ortogonální mřížka neumožňuje. Příkladem informované strategie využívající tuto heuristiku je hladové hledání nejlepšího souseda (*Greedy Best-First Search*), které k expanzi vždy vybírá uzel s nejnižší hodnotou $h(n)$. Hledání je tím agresivně směřováno k cíli, avšak za cenu ztráty optimality – nalezená cesta nemusí být nejkratší možnou, narazí-li algoritmus na překážky [RN21].

2.2.3 Lokální vyhledávání a horolezecký algoritmus (Hill Climbing)

Zatímco klasické prohledávací algoritmy (včetně DFS či Best-First Search) si udržují v paměti hranice prozkoumaného prostoru (*Open Set*) a umožňují návrat (*backtracking*), existují situace, kdy je ukládání předchozích stavů výpočetně či paměťově nežádoucí. Pro tyto případy slouží algoritmy lokálního vyhledávání (*Local Search*) [RN21].

Typickým zástupcem je horolezecký algoritmus (*Hill Climbing*). Jedná se o iterativní metodu, která udržuje pouze jeden aktuální stav (uzel). V každém kroku algoritmus vyhodnotí své bezprostřední sousedy a provede nezvratný přechod do toho souseda, který nejvíce minimalizuje (či maximalizuje) zadanou heuristickou funkci.

Zásadní vlastností algoritmu Hill Climbing je absence paměti a nemožnost kroku zpět. V důsledku toho je algoritmus vysoce náchylný k uvíznutí v lokálním minimu

– situaci, kdy žádný ze sousedních uzlů nemá lepší heuristické ohodnocení, avšak globálního cíle ještě nebylo dosaženo. V takovém případě algoritmus selhává. Navzdory tomuto riziku je metoda pro svou minimální paměťovou a výpočetní náročnost využívána jako základní stavební blok pro komplexnější generativní systémy a optimalizační úlohy [RN21].

2.3 Ant Colony Optimization (ACO)

Algoritmus *Ant Colony Optimization* (ACO) je pravděpodobnostní metaheuristická technika, kterou v devadesátých letech 20. století zavedl Marco Dorigo [DG97]. Tento přístup spadá do širší rodiny algoritmů rojové inteligence (*Swarm Intelligence*) a byl primárně navržen pro řešení těžkých kombinatorických optimalizačních úloh, jako je problém obchodního cestujícího (TSP – *Traveling Salesman Problem*). Následný vývoj však prokázal jeho vysokou flexibilitu a úspěšnost i v oblastech dynamického směřování a hledání optimálních cest v diskrétních grafech [DS04; Liu+26].

2.3.1 Biologická inspirace a feromonová komunikace

Základním inspiračním zdrojem ACO je chování reálných mravenců při hledání potravy. Ačkoliv je jedinec mravenčí kolonie vybaven pouze velmi omezenou inteligencí a zrakem, kolonie jako celek dokáže spolehlivě nalézt nejkratší cestu mezi mraveništěm a zdrojem potravy. Tento fenomén je umožněn nepřímou formou komunikace zvanou *stigmergie*.

Mravenci při pohybu prostředím vylučují chemickou stopu – feromon. Když se mravenec rozhoduje, jakým směrem se vydat, s vyšší pravděpodobností volí cestu, na které je koncentrace feromonu silnější. Kratší cesty jsou prozkoumávány rychleji, což vede k rychlejší akumulaci feromonu na těchto úsecích v porovnání s delšími objíždkami. Tato pozitivní zpětná vazba (*positive feedback loop*) nakonec způsobí, že většina kolonie konverguje k optimální trase [DS04].

2.3.2 Princip algoritmu a umělí agenti

V algoritmickém prostředí je tento biologický proces modelován pomocí tzv. umělých mravenců (agentů), kteří iterativně prozkoumávají stavový prostor úlohy, v případě této práce reprezentovaný 3D mřížkou buněk ($Cell[i, j]$). Při konstrukci cesty se agent nacházející se v uzlu i rozhoduje o přechodu do sousedního uzlu j na základě stochastického pravidla, které zohledňuje dva klíčové faktory:

1. **Feromonovou stopu (τ_{ij}):** Hodnota reprezentující historickou "úspěšnost" daného přechodu napříč předchozími iteracemi.

2. **Heuristickou informaci (η_{ij}):** Reprezentuje znalost o výhodnosti přechodu z aktuálního uzlu i do sousedního uzlu j . V předkládané implementaci je tato hodnota definována na základě manhattanské vzdálenosti mezi zkoumaným uzlem j a cílovou lokací. Matematicky lze tuto závislost vyjádřit jako převrácenou hodnotu vzdálenosti. Čím blíže je potenciální další krok k cíli, tím vyšší je jeho heuristické ohodnocení η_{ij} .

Spojením těchto parametrů algoritmus vyhodnocuje pravděpodobnost, s jakou agent daný krok zvolí. Tento přístup umožňuje algoritmu balancovat mezi *exploitací* (využíváním již známých dobrých cest silně pokrytých feromonem) a *explorací* (prozkoumáváním nových, potenciálně lepších tras na základě heuristiky) [DG97].

2.3.3 Aktualizace stop: Odpařování a posilování

Klíčovým mechanismem zamezujícím předčasnou konvergenci algoritmu k lokálnímu optimu je odpařování feromonu (*Pheromone Evaporation*). Po dokončení každé iterace (kdy agenti dorazí do cíle nebo selžou), je koncentrace feromonu na všech hranách grafu plošně snížena o definovaný koeficient odpařování. Tento krok zajišťuje, že systém "zapomíná" špatná rozhodnutí a umožňuje adaptaci na případné změny v topologii [DS04].

Následně probíhá fáze posílení (*Pheromone Deposit*), při které úspěšní agenti přidají na projité hrany nové množství feromonu, obvykle nepřímo úměrné délce nalezené cesty. Trasy, které spojily start s cílem v menším počtu kroků, jsou tak zvýhodněny pro budoucí iterace.

2.3.4 Aplikace v generování herních prostředí (Integrace s WFC)

Na rozdíl od klasického problému obchodního cestujícího, kde je cílem navštívit všechny uzly sítě, je využití ACO v předkládaném architektonickém řešení modifikováno pro problém hledání cesty v nejistém prostředí (*Path Planning under Uncertainty*).

V navrženém hybridním frameworku mravenci neprocházejí hotovým bludištěm, ale prozkoumávají prostor nacházející se ve stavu kvantové superpozice. Agenti při výběru dalšího kroku ověřují dostupnost kompatibilních socketů v superpozici sousedních buněk. Jakmile ACO vrstva nalezne funkční páteřní cesty propojující počáteční a cílový bod, zanechá v příslušných buňkách feromonové hodnoty.

V předkládané implementaci vstupuje výstup ACO vrstvy do WFC fáze jako pevné omezení (*hard constraint*): garantovaná průchozí trasa je předpočítána před spuštěním WFC smyčky a slouží jako strukturální předpoklad pro kolaps okolního prostředí. Tento přístup je podrobněji popsán v sekci 3.3.3.

Architektura systému

3

Systém je rozdělen do tří vrstev na principu separace zodpovědností:

- **Datová vrstva** definuje stavební bloky prostředí (TileData, TileVariant) a udržuje stavovou mřížku buněk (Cell).
- **Logická vrstva** obsahuje generativní solvery – WFCSolver jako základ, GLSHybridSolver pro lokální prohledávání a ACOHybridSolver pro optimalizaci páteřních tras. Izolace této vrstvy umožňuje přidávat nové solvery bez zásahu do zbytku systému.
- **Prezentační vrstva** transformuje výsledky solverů do 3D scény – instancuje modely na vypočtené souřadnice.

3.1 Reprezentace prostoru

Prostředí je trojrozměrná mřížka buněk. Každá buňka začíná ve stavu superpozice – obsahuje množinu všech přípustných modulů. Generování postupně redukuje tuto množinu aplikací omezujících podmínek (CSP), přičemž kolaps jedné buňky vynucuje omezení u sousedů. Míra neurčitosti buňky je vyjádřena entropií:

$$H(c) = |S_c| \quad (3.1)$$

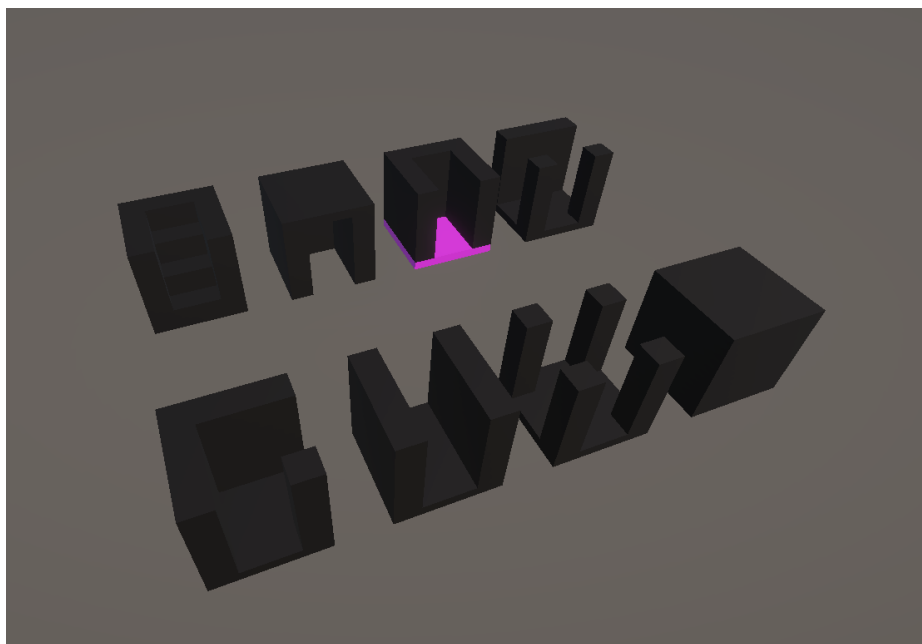
3.2 Sockety a rotace modulů

Kompatibilita sousedních modulů je řešena přes textové identifikátory (sockety) – každý modul má šest socketů pro každý ortogonální směr. Dva moduly mohou sousedit, pokud se jejich protilehlé sockety shodují.

Rotované varianty modulů jsou odvozovány dynamicky při inicializaci podle vzorce:

$$S_{new}[i] = S_{orig}[(i - r + 4) \pmod{4}] \quad (3.2)$$

kde i je index směru a r počet otočení o 90° .



Obrázek 3.1: Katalog stavebních modulů systému. Horní řada zleva: *Stairs_Bottom*, *Stairs_Top*, *Start-Finish*, *T-Junction*. Dolní řada zleva: *Corner*, *Corridor*, *Cross*, *Solid-Block*.

Rozlišují se dva koncepty: *TileData* představuje abstraktní definici typu modulu – jeho geometrii a socketovou šablonu – zatímco *TileVariant* reprezentuje konkrétní přípustnou variantu odvozením rotace $r \in \{0, 1, 2, 3\}$ z příslušného *TileData*. Množina přípustných variant buňky S_c je tedy podmnožinou všech *TileVariant* objektů dostupných v systému.

3.3 Generativní solvery

Systém obsahuje tři solvery, které sdílejí společné jádro *WFCSolver* a liší se strategií výběru páteřní trasy.

3.3.1 WFCSolver

Základní solver implementuje algoritmus Wave Function Collapse nad trojrozměrnou mřížkou. Pro výběr další buňky ke kolapsu je využívána minimální halda *CellMinHeap* s líným mazáním (*lazy deletion*): priorita buňky je zmrazena v době vložení a při extrakci je záznam zahozen, pokud uložená priorita neodpovídá aktuální hodnotě $H(c) = |S_c|$. Zkolabované buňky jsou při dequeue zahazovány na základě příznaku *IsCollapsed*, čímž je zaručeno, že halda vždy vrátí buňku s nejnížší entropií mezi dosud nezkolabovanými buňkami. Konkrétní modul je následně vybrán stochasticky z množiny aktuálně přípustných variant.

Okrajové buňky mřížky jsou před spuštěním inicializovány pevnými *wall* sockety, čímž je zajištěna uzavřenost prostředí. Při detekci kontradikce – tedy situace, kdy se množina přípustných variant některé buňky vyprázdní – solver signalizuje selhání nadřazené řídicí logice, která proces opakuje. Mechanismus snapshotů slouží výhradně ACO vrstvě pro nedestruktivní ověřování kandidátních tras.

3.3.2 GLSHybridSolver

Solver GLSHybridSolver implementuje variantu horolezeckého algoritmu (Hill Climbing) adaptovanou pro konstrukci páteřní trasy.

Solver nejprve zkonstruuje garantovanou průchozí trasu mezi dvěma body, poté předá řízení WFCSolver pro zaplnění zbývajících prostředí.

Crawler v každém kroku identifikuje sousedy aktuální buňky s kompatibilním *path* nebo *stairs_up* socketem. Pro třídění nejvýše šesti sousedních kandidátů je využita euklidovská vzdálenost namísto manhattanské. Tato volba je odůvodněna kontextem použití: vzdálenost slouží výhradně jako lokální komparátor malého konstantního počtu sousedů, nikoliv jako navigační cena v rozsáhlém prostoru. Výpočetní náklad odmocniny je proto zanedbatelný a obě metriky vedou na tomto měřítku ke shodné relaci uspořádání. Vybraný soused je okamžitě kolapsován voláním ForceCollapse, přičemž jsou preferovány varianty s nejvýše dvěma path/stairs-sockety jako strukturálně čistší. Maximální počet kroků crawleru je omezen na $2 \times \text{totalCells}$.

3.3.3 ACOHybridSolver

Solver využívá ACO vrstvu jako zdroj pevných omezení (*hard constraints*): před spuštěním WFC fáze je předpočítána garantovaná průchozí trasa, která vstupuje do kolapsu jako strukturální předpoklad. Feromonové hodnoty tedy neovlivňují výběr variant uvnitř WFC smyčky, ale deterministicky určují globální topologii průchozí osy.

Solver k nalezení optimální trasy prohledává feromonově ohodnocenou mřížku pomocí ACO. Každý agent volí další buňku probabilisticky na základě aktuální hladiny feromonu a heuristiky definované jako převrácená Manhattanská vzdálenost k cíli. Konkrétní parametry přechodového pravidla jsou diskutovány v sekci testování.

Před finálním kolapsem je každá kandidátní trasa ověřena pomocí SaveSnapshot/RestoreSnapshot – pokud trasa způsobí kontradikci ve WFC, jsou její feromony penalizovány. Úspěšná trasa je poté předána WFCSolver ke zaplnění zbytku mřížky.

Implementace algoritmů

4

Tato kapitola popisuje způsob, jakým jsou jednotlivé algoritmy adaptovány pro potřeby systému. Důraz je kladen na odlišnosti oproti jejich klasickým formulacím, nikoliv na detaily implementace.

4.1 Wave Function Collapse

V původní formulaci Maxima Gumina [Gum16] algoritmus WFC pracuje s dvourozměrnými bitmapami, přičemž pravidla sousednosti jsou odvozována automaticky analýzou vzorového obrázku. Výsledkem je generátor textur, který dokáže syntetizovat nový obsah zachovávající lokální vzory předlohy.

Tato implementace vychází z odlišného přístupu, který popisuje Kleineberg [Kle19] – místo automatické extrakce pravidel z textury jsou stavební bloky a jejich kompatibilita definovány ručně pomocí socketů. Algoritmus tak operuje nad trojrozměrnou mřížkou ručně navržených modulů namísto dvourozměrného pole pixelů. Karth a Smith [KS17] ukazují, že WFC lze obecně chápat jako formu řešení problému splnění omezujících podmínek (CSP), což odpovídá způsobu, jakým je propagace omezení implementována zde.

Výběr buňky ke kolapsu pomocí minimální haldy a náhodný výběr modulu z přípustných variant odpovídají standardní formulaci algoritmu. Odlišnost spočívá v rozšíření o snapshotový mechanismus zásobníku změn, který není součástí původní formulace algoritmu. Tento mechanismus neslouží k backtrackingu uvnitř WFC smyčky – při detekci kontradikce solver standardně signalizuje selhání. Snapshotový systém je namísto toho využit ACO vrstvou pro nedestruktivní ověřování kandidátních tras před jejich aplikací jako pevných omezení.

4.2 Greedy Local Search

Greedy local search není v tomto systému použit jako samostatný generátor prostředí, ale jako mechanismus garantující existenci průchozí trasy mezi dvěma body

před spuštěním WFC. Tato role vychází z limitace samotného WFC – algoritmus optimalizuje globální konzistenci socketů, ale průchozí trasu jako takovou nezaručuje.

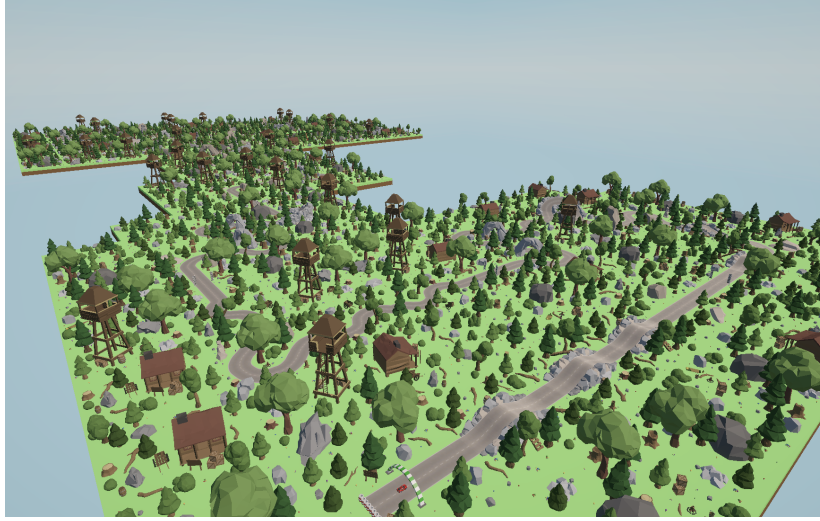
Crawler postupuje hladově od startovního bodu k cíli: v každém kroku vybere nejbližšího souseda s kompatibilním path socketem a okamžitě ho zkolabuje na modul odpovídající trase. Teprve po úspěšném sestavení trasy je předáno řízení WFC solveru, který zaplní zbývající buňky mřížky. Výsledkem je prostředí, kde průchozí trasa existuje vždy, zatímco okolní struktura je generována standardním WFC.

Zásadní odlišností od klasického heuristického prohledávání (Best-First Search) je absence zásobníku otevřených uzlů a mechanismu návratu. Crawler udržuje pouze aktuální pozici, čímž se blíží paradigmatu Hill Climbing – slepá ulička znamená okamžité selhání a restart celého procesu řídicí logikou. Tato vlastnost je přijatelná, neboť crawler neprohledává hotové bludiště, ale kolabuje buňky v průběhu pohybu. Zpětný krok by vyžadoval zrušení již aplikovaných omezení v mřížce, což je výpočetně velmi náročné.

Pro třídění kandidátních sousedů je využita euklidovská vzdálenost namísto manhattanské. Tato volba je odůvodněna kontextem použití – vzdálenost slouží výhradně k seřazení malého počtu sousedních buněk (nejvýše šesti), nikoliv jako navigační cena v rozsáhlém prostoru. Výpočetní náklad odmocniny je v tomto kontextu zanedbatelný.

4.2.1 Praktické nasazení v externím projektu

Algoritmus GLSHybridSolver byl nasazen v rámci skupinového projektu předmětu KIV/PH jako generátor závodní tratě na dvojrozměrné mřížce. Oproti konfiguraci popsané v předchozích sekcích se instance liší ve třech aspektech: cílová buňka není pevně stanovena – crawler terminuje po dosažení požadované délky trasy, velikost mřížky je dynamicky odvozena od této délky a kromě startovní a cílové buňky jsou do trasy vkládány checkpointy na každé šesté pozici. Integrita tratě je zajištěna podmínkou zamítající každý krok, který by vytvořil zkratku mezi vzdálenými úseky trasy. Toto nasazení dokládá přenositelnost algoritmu mimo původní 3D kontext na úlohy s odlišnou topologií a dynamicky definovaným cílem.



Obrázek 4.1: Ukázka vygenerované závodní tratě algoritmem GLSHybridSolver v projektu předmětu KIV/PH.

4.3 Ant Colony Optimization

Klasická formulace ACO, kterou popisují Dorigo a Gambardella [DG97] a dále Dorigo a Stützle [DS04], je primárně určena pro řešení problému obchodního cestujícího (TSP) na plném grafu. Agenti probabilisticky volí další uzel j ze množiny přípustných sousedů $\mathcal{N}(i)$ podle přechodového pravidla:

$$P_{\text{TSP}}(i \rightarrow j) = \frac{\tau_{ij}^{\alpha} \cdot \eta_{ij}^{\beta}}{\sum_{k \in \mathcal{N}(i)} \tau_{ik}^{\alpha} \cdot \eta_{ik}^{\beta}} \quad (4.1)$$

kde τ_{ij} je hladina feromonu na hraně (i, j) , η_{ij} je heuristická informace o výhodnosti přechodu (v TSP typicky převrácená hodnota délky hrany), a α, β jsou parametry vyvažující vliv feromonu a heuristiky [DS04].

Adaptace pro tento systém přináší tři hlavní odlišnosti oproti klasické formulaci. Za prvé, místo plného grafu agenti prohledávají trojrozměrnou voxelovou mřížku, kde přechody existují pouze mezi sousedními buňkami. Za druhé, feromony jsou v implementaci přiřazeny *buňkám* namísto hranám — hodnota $\tau(j)$ reprezentuje feromonovou hladinu cílového uzlu j , čímž se přechodové pravidlo zjednodušuje na:

$$P(i \rightarrow j) = \frac{\tau(j)^{\alpha} \cdot \eta(j)^{\beta}}{\sum_{k \in \mathcal{N}(i)} \tau(k)^{\alpha} \cdot \eta(k)^{\beta}} \quad (4.2)$$

kde $\eta(j) = (d(j, g) + \epsilon)^{-1}$ je heuristická hodnota odvozená z manhattanské vzdálenosti k cíli g a ϵ je malá konstanta zabráňující dělení nulou. Za třetí, každá kandi-

dátní trasa nalezená kolonií je před aktualizací feromonů ověřena simulací v WFC – WFC validace tak funguje jako fitness funkce směřující kolonii k trasám strukturně kompatibilním s okolním prostředím. Klasické hodnoty parametrů navržené pro TSP [DS04] se v tomto kontextu ukázaly jako nevhodné a vyžadovaly empirické přeladění, jehož výsledky jsou diskutovány v kapitole testování.

Aktualizace feromonové matice po dokončení každé iterace probíhá ve dvou krocích. Nejprve je aplikováno plošné odpařování s koeficientem $\rho \in (0, 1)$:

$$\tau(j) \leftarrow (1 - \rho) \cdot \tau(j) \quad (4.3)$$

Následně každý úspěšný agent posílí feromony na projitých buňkách o hodnotu nepřímo úměrnou délce nalezené trasy L :

$$\tau(j) \leftarrow \tau(j) + \frac{1}{L} \quad (4.4)$$

Kratší trasy jsou tak systematicky zvýhodňovány pro následující iterace. Koeficient $\rho = 0,15$ byl stanoven empiricky na základě pilotního testování popsaného v kapitole 6.

Řídící logika a uživatelské rozhraní

5

Tato kapitola popisuje vrstvu organizace, která koordinuje generativní proces, ošetřuje chybové stavy a zprostředkovává interakci s uživatelem.

5.1 Řídící logika

Řídící logika zastává roli organizátoru celého generativního cyklu. Po obdržení parametrů od uživatele inicializuje mřížku, předá řízení zvolenému solveru a v případě selhání proces opakuje. Maximální počet pokusů je dynamicky škálován podle velikosti mřížky:

$$n_{\max} = \text{clamp}(20 + \lfloor N/2 \rfloor, 20, 500) \quad (5.1)$$

kde N označuje celkový počet buněk mřížky. Toto škálování reflektuje skutečnost, že pravděpodobnost kontradikce roste s velikostí prostoru, a zajišťuje, že generátor věnuje rozsáhlejším konfiguracím proporcionálně více pokusů.

Výjimku tvoří `ACOHybridSolver`, jemuž jsou přiděleny nejvýše tři pokusy. Tento limit vychází z experimentálně ověřené spolehlivosti solveru – při třech pokusech dosahuje ACO na testovaných konfiguracích mizivé pravděpodobnosti celkového selhání, jak je doloženo v kapitole testování.

5.2 Uživatelské rozhraní a ochrana proti chybným vstupům

Uživatelské rozhraní zpřístupňuje sadu předvolených konfigurací mřížky pokrývající rozsah od malých dvojrozměrných map ($10 \times 1 \times 10$ buněk) po rozsáhlé trojrozměrné konfigurace ($30 \times 5 \times 30$ buněk), a umožňuje volbu mezi třemi solvery.

Součástí rozhraní je systém dynamických varování, která jsou zobrazena kontextově na základě zvolené kombinace solveru a konfigurace mřížky. Varování upozorňují zejména na kombinace náchylné ke kombinatorické explozi – například

volba hladového crawleru na rozsáhlé dvojrozměrné mřížce ($50 \times 1 \times 50$ buněk) je označena jako nevhodná s doporučením použití ACO solveru. Tato opatření nevyžadují od uživatele žádnou potvrzovací akci a slouží výhradně jako informační vrstva před spuštěním generování.

Po selhání všech pokusů ACOHybridSolveru je automaticky aktivována vizualizace feromonové matice. Jednotlivé uzly mřížky jsou vykresleny jako prostorové objekty, jejichž velikost a barevné kódování – interpolované od studené (nízká koncentrace feromonu) po teplou (vysoká koncentrace) – reflektují výslednou distribuci feromonů. Tato vizualizace umožňuje diagnostikovat oblasti mřížky, v nichž agenti opakovaně selhávali při hledání buildable trasy, a slouží primárně jako diagnostický nástroj při vývoji a ladění parametrů kolonie. Za standardních provozních podmínek je pravděpodobnost selhání všech pokusů ACO solveru vzhledem k jeho experimentálně ověřené spolehlivosti velmi nízká.

The image shows a user interface for a 'WFC based generator'. It is divided into three main sections: 1. Grid size, 2. Algorithm, and 3. Seed (empty = random). Section 1 has six buttons for grid sizes: Small 3D (10x3x10), Small 2D (10x1x10), Medium 3D (20x4x20), Medium (25x1x25), Large 3D (30x5x30), and Large 2D (50x1x50). Section 2 has three buttons for algorithms: GLS Crawler, ACO Hybrid, and Pure WFC. Section 3 has a text input field for a seed and a 'Clear' button. At the bottom are two large buttons labeled 'GENERATE' and 'EXIT'.

WFC based generator

1. Grid size:

Small 3D (10x3x10) Small 2D (10x1x10)

Medium 3D (20x4x20) Medium (25x1x25)

Large 3D (30x5x30) Large 2D (50x1x50)

2. Algorithm:

GLS Crawler

ACO Hybrid

Pure WFC

3. Seed (empty = random):

GENERATE

EXIT

Obrázek 5.1: Uživatelské rozhraní pro výběr algoritmu a velikost mřížky

5.3 Režim izolované trasy

V případě, kdy hladový crawler úspěšně sestaví průchozí trasu, avšak následné spuštění `WFCsolveru` skončí kontradikčním stavem, systém přejde do režimu izolované trasy (*Path Only Mode*). V tomto režimu jsou vizualizovány výhradně buňky tvořící zkolabovanou páteřní trasu; okolní buňky mřížky zůstávají v nezkolabovaném stavu a nejsou instancovány.

Tento režim je funkční výhradně pro jednopatrové dvojrozměrné konfigurace a nepředstavuje plnohodnotné řešení generování prostředí. Jeho účelem je demonstrace funkčnosti crawleru v izolaci od WFC vrstvy a poskytnutí částečného výstupu v případech, kdy úplná generace selhává. V trojrozměrných konfiguracích je místo

tohoto režimu vráceno selhání a iniciován nový pokus.

5.4 Minimapa

Komponenta `MinimapController` zajišťuje průběžnou vizualizaci generované mapy formou 2D ortografického průřezu aktuálního patra. Pracuje nezávisle na generačním cyklu – z `MapGenerator` čte výhradně veřejné vlastnosti `tileSize` a `mapFloors` a nemá přístup k internímu stavu mřížky.

Aktuální patro f je odvozováno z vertikální pozice pozorovatele y_{obs} podle vztahu:

$$f = \text{clamp}\left(\left\lfloor \frac{y_{\text{obs}}}{s} \right\rfloor, 0, F - 1\right) \quad (5.2)$$

kde s je výška dlaždice a F je celkový počet pater. Ořez na interval $\langle 0, F - 1 \rangle$ zabraňuje neplatnému indexování při pohybu pozorovatele mimo hranice generované mřížky.

Změna f vyvolá přepočítání pozice a parametrů ortografické kamery. Průřez se tak dynamicky aktualizuje tak, aby odpovídal patru, v němž se pozorovatel právě nachází. Poloha značky pozorovatele na minimapě je udržována v reálném čase projekcí jeho světových souřadnic x a z do roviny aktuálního průřezu.

Testování, výsledky a diskuze

6

6.1 Metodika testování

Všechny experimenty byly prováděny jako opakované měření na pevně zvolených konfiguracích mřížky s cílem statisticky popsat chování jednotlivých solverů. Sledovanými veličinami jsou celkový čas generace (v milisekundách), počet nutných pokusů do úspěšného kolapsu, délka výsledné páteřní trasy (v počtu buněk) a míra úspěšnosti (podíl úspěšných generací vůči celkovému počtu pokusů, vyjádřený v procentech).

Výsledky jsou uváděny jako průměry přes n nezávislých běhů, přičemž n se pohybuje od 5 do 100 v závislosti na výpočetní náročnosti konfigurace a cíli experimentu. Generátor pseudonáhodných čísel není seedován fixní hodnotou a výsledky tak nejsou deterministicky reprodukovatelné. Tato vlastnost představuje jednu z koncepčních limitací, která je podrobněji diskutována v sekci 6.8.

Testovací konfigurace pokrývají rozsah od malých plochých mřížek ($10 \times 1 \times 10$ buněk) po komplexnější trojrozměrné prostory ($30 \times 5 \times 30$ buněk). Pro zátěžové jsou použity konfigurace až $75 \times 5 \times 75$ buněk, které leží mimo běžný provozní rozsah systému. Testy byly spouštěny v prostředí editoru Unity na MacBooku Air s M2 čipem a 16GB RAM.

6.2 Optimalizace parametrů ACO

Parametr α byl po úvodním pilotním testování fixován na hodnotu $\alpha = 1,0$, čímž feromonová složka vstupuje do přechodového pravidla lineárně. Optimalizace se dále soustředí na parametr β , jehož vliv na poměr exploitace a explorační se ukázal jako dominantní.

Parametr β přechodového pravidla ACO (viz rovnice v sekci o implementaci) řídí relativní váhu heuristické složky vůči feromonové stopě. Volba nevhodné hodnoty β vede buď k nadbytečné exploraci bez konvergence, nebo k předčasnému uvíznutí.

nutí kolonie v lokálním optimu. Parametr byl proto experimentálně optimalizován třífázovým sweepe testem přes různé hodnoty.

6.2.1 Fáze 1 — hrubé prohledávání

První fáze prohledávala rozsah $\beta \in \{1,0; 2,0; 3,5; 5,0; 7,0\}$ s parametry kolonie (colony = 22, iter = 40) na mřížce $30 \times 5 \times 30$. Každá hodnota β byla vyhodnocena na základě míry úspěšnosti kolonii dosáhnout cíle (tabulka 6.1).

β	Úspěšnost (%)
1,0	0
2,0	0
3,5	20
5,0	80
7,0	100

Tabulka 6.1: Fáze 1 beta sweepu — mřížka $30 \times 5 \times 30$, colony = 22, iter = 40

Při hodnotách $\beta \leq 2,0$ mravenci bloudí bez směru — feromonová složka dominuje nad heuristikou a kolonie konverguje k lokálním smyčkám namísto k cíli. Výsledek potvrzuje, že ve trojrozměrném diskrétním prostoru bez přímých diagonálních přechodů je vysoká hodnota β nezbytnou podmínkou funkčního navádění agentů.

6.2.2 Fáze 2 — jemné prohledávání

Druhá fáze prohledávala rozsah $\beta \in [6,0; 8,0]$ s krokem 0,1, přičemž každá hodnota byla vyhodnocena na 20 nezávislých bžích (tabulka 6.2).

β	Úspěšnost (%)	Průměrná délka cesty
6,0	40	94,0
6,5	80	85,0
7,0	60	80,3
7,5	80	83,0
7,6	100	69,8
7,7	100	71,0
7,8	80	75,0
7,9	100	68,6
8,0	60	75,7

Tabulka 6.2: Fáze 2 beta sweepu — 20 běhů na hodnotu, mřížka $30 \times 5 \times 30$

Z tabulky je patrné, že volba β je důležitá — rozsah funkčních hodnot je úzký a chování kolonie je na jeho přesném nastavení citlivé. Nižší hodnoty ($\beta \leq 7,0$)

ponechávají kolonii příliš mnoho prostoru pro exploraci, čímž konvergence k cíli zpomaluje a úspěšnost klesá.

Pokles úspěšnosti pro $\beta \geq 8,0$ lze přičíst dvěma typům topologických překážek, k jejichž obejití příliš vysoké β nedává kolonii dostatečný prostor.

Prvním typem je *socketová slepá ulička*: buňka sousedící s aktuální pozicí agenta je z hlediska Manhattanské vzdálenosti výhodná, ale žádná z jejích dostupných variant neobsahuje výstupní socket kompatibilní s požadovaným pokračováním trasy. Agent takovou buňku obsadí a teprve až zpětná WFC simulace odhalí kontradikci — trasa je penalizována.

Druhým typem je *feromonová smyčka*: kolonie kumuluje feromony na buildable, ale na výrazně delší trase (typicky podél okraje mřížky), protože kratší přímá cesta skrze hustě osázenou část prostoru opakovaně způsobuje kontradikci. Vysoké β pak tlačí nové agenty k cíli právě přes tuto suboptimální smyčku, namísto aby explorací hledali kratší alternativu. Příklad takové smyčky je zachycen na obrázku 6.1, kde pohled shora i minimapa znázorňuje cestu s délkou 193 buněk, kde se agenti snažili o průchod středem a došlo k zacyklení.



Obrázek 6.1: Příklad feromonové smyčky: Výsledná páteřní cestu zvýrazněná žlutě. Cesta je buildable — kolonie ji fixovala feromonem, přestože je suboptimální.

6.2.3 Fáze 3 — verifikace

Třetí fáze ověřila robustnost výsledku ve dvou nezávislých sadách po 20 bězích pro $\beta \in \{7,8; 7,9; 8,0; 8,1; 8,2\}$. Hodnoty 7,8–8,1 jsou statisticky těsně blízké. Hodnota $\beta = 7,9$ byla zvolena jako praktický kompromis: dosahuje konzistentní 100 %

úspěšnosti napříč oběma verifikačními sadami a zároveň vykazuje nejnižší průměrnou délku cesty (68,6 buněk) ze všech testovaných hodnot se stejnou úspěšností. Přesné statistické odlišení od $\beta \in \{7,6; 7,7\}$ by vyžadovalo řádově větší počet běhů a detailní zkoumání parametrů není cílem práce.

Je třeba zdůraznit, že optimalizace byla prováděna výhradně na mřížce $30 \times 5 \times 30$. Přenositelnost nalezené hodnoty β na konfigurace výrazně odlišných rozměrů nebo jiné topologické charakteristiky mřížky nebyla systematicky ověřena a vyžadovala by další testování.

6.3 Výkonnostní srovnání výběru entropie ve WFC

Prvotní implementace algoritmu WFC vybírala buňku s minimální entropií lineárním prohledáváním celé mřížky v každém kroku — složitost $O(N)$ na výběr. Nahrazení lineárního prohledávání minimální haldou s líným mazáním (*lazy deletion*) snižuje tuto složitost na $O(\log N)$. Srovnání obou přístupů bylo provedeno na pěti konfiguracích mřížky, každá s 100 nezávislými běhy (tabulka 6.3).

Konfigurace	Buněk	Lineární (ms)		Minimální halda (ms)	
		Celkem	Entropie	Celkem	Entropie
$10 \times 3 \times 10$	300	8,68	0,00	9,61	0,00
$15 \times 3 \times 15$	675	21,09	1,00	24,06	0,00
$20 \times 4 \times 20$	1 600	64,17	9,67	67,78	0,00
$25 \times 4 \times 25$	2 500	116,42	25,96	108,70	0,99
$30 \times 5 \times 30$	4 500	268,03	81,59	212,44	1,99

Tabulka 6.3: Srovnání výběru entropie — lineární prohledávání vs. minimální halda

Na nejrozsáhlejší testované konfiguraci (4 500 buněk) klesla doba strávená výběrem entropie z 81,59 ms na 1,99 ms — zrychlení přibližně 41násobné. Podíl výběru entropie na celkovém čase klesl z 30,4 % na 0,9 %, čímž byl tento bottleneck eliminován. Celková doba generace se snížila z 268 ms na 212 ms (úspora ≈ 21 %), přičemž zbývající čas tvoří propagace omezení a kolaps, které jsou v obou variantách identické.

Na konfiguracích do 1 600 buněk je nadbytečná réžie haldy ($O(\log N)$ operace místo přímého přístupu) srovnatelná s lineárním přístupem, takže celkové časy jsou v tomto rozsahu mírně vyšší pro variantu s haldou. Přínos minimální haldy se projevuje až od přibližně 2 500 buněk a s rostoucí velikostí mřížky dále roste.

6.4 Srovnání solverů GLS a ACO

GLSHybridSolver a ACOHybridSolver byly porovnány na šesti konfiguracích pokrývajících jak dvourozměrné, tak trojrozměrné mřížky. Každá kombinace solveru a konfigurace byla vyhodnocena na 20 nezávislých bězích (tabulka 6.4).

Konfigurace	Solver	Čas (ms)	Pokusy	Délka cesty	Úspěšnost (%)
$10 \times 1 \times 10$	GLS	11,7	12,85	31,8	100
$10 \times 1 \times 10$	ACO	18,8	1,05	19,3	100
$25 \times 1 \times 25$	GLS	597,8	216,5	102,7	75
$25 \times 1 \times 25$	ACO	62,0	1,00	51,5	100
$50 \times 1 \times 50$	GLS	3 681,4	500,0	—	0
$50 \times 1 \times 50$	ACO	185,0	1,05	169,5	100
$10 \times 3 \times 10$	GLS	25,2	14,75	34,2	100
$10 \times 3 \times 10$	ACO	38,6	1,00	21,4	100
$20 \times 4 \times 20$	GLS	362,5	88,75	78,5	100
$20 \times 4 \times 20$	ACO	200,3	1,00	45,1	100
$30 \times 5 \times 30$	GLS	2 462,4	331,3	133,8	60
$30 \times 5 \times 30$	ACO	873,2	1,10	74,0	100

Tabulka 6.4: Srovnání GLS a ACO solverů — 20 běhů na kombinaci

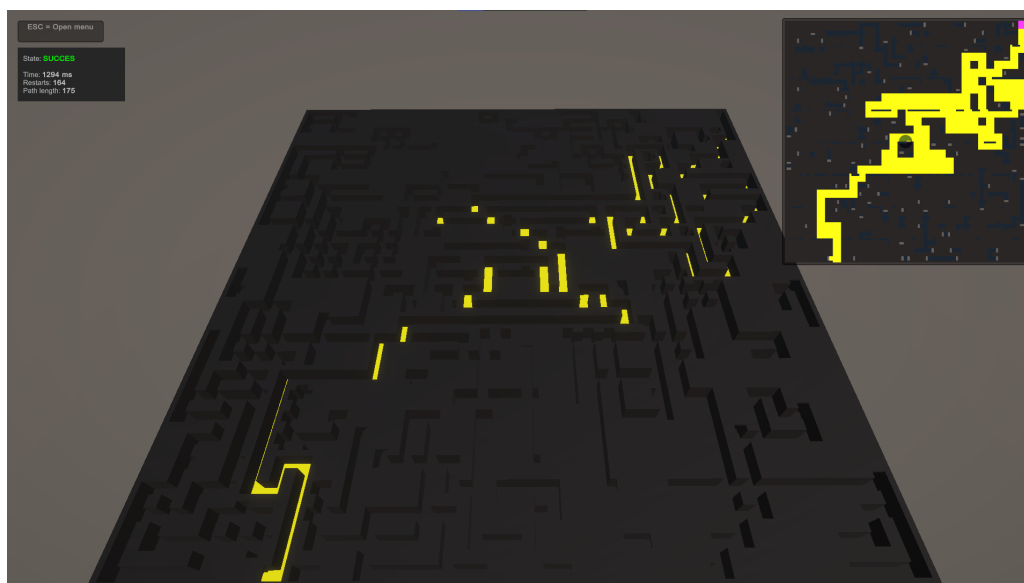
Z výsledků vyplývají tři strukturální rozdíly mezi oběma přístupy.

Spolehlivost. Hladový crawler bez mechanismu návratu je náchylný k uvíznutí v lokálním minimu — na konfiguraci $50 \times 1 \times 50$ vyčerpал veškeré dostupné pokusy (500) bez jediného úspěchu. Příčina nebyla v rámci této práce detailně prozkoumána a vyžaduje další analýzu. ACO solver naproti tomu na téže konfiguraci uspěl v průměru na první pokus s 100% úspěšností. Na konfiguraci $30 \times 5 \times 30$ dosáhl crawler 60% úspěšnosti při průměrně 331 pokusech, zatímco ACO si vystačil průměrně s 1,1 pokusem. Vizuální srovnání těchto konfigurací je zachyceno na obrázcích 6.2 a 6.3.

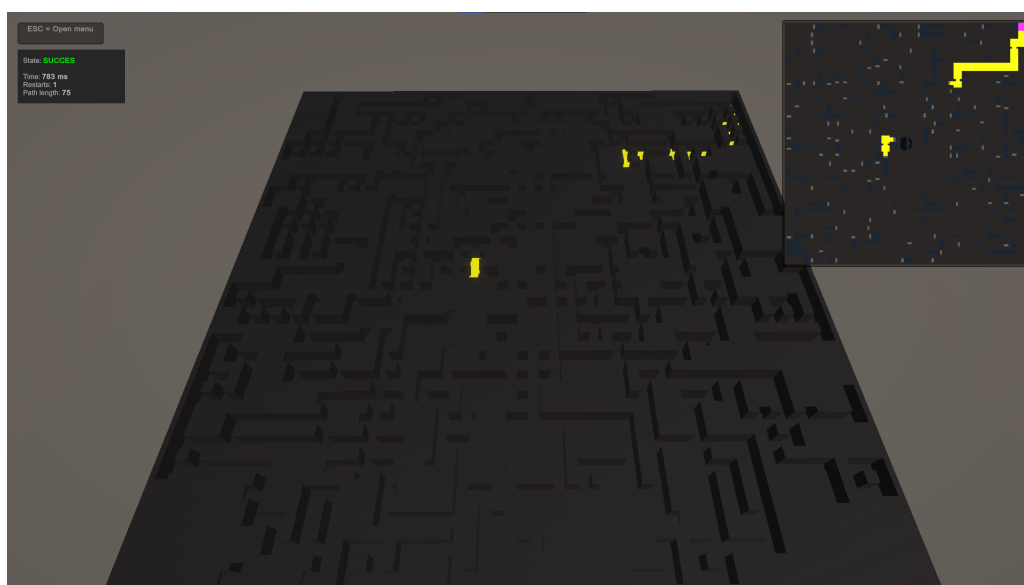
Délka cesty. ACO generuje výrazně kratší páteřní trasy. Na trojrozměrných konfiguracích je cesta ACO solveru o 38–45 % kratší než cesta crawleru: na $10 \times 3 \times 10$ je rozdíl 34,2 vs. 21,4 buněk, na $20 \times 4 \times 20$ pak 78,5 vs. 45,1 buněk. Kratší cesta přímo odpovídá strukturálně čistšímu prostředí s menší pravděpodobností kontradikce při dokončení WFC fáze.

Výpočetní čas. Crawler je pro malé mřížky rychlejší, avšak jeho celkový čas (včetně opakovaných pokusů) dramaticky roste s velikostí prostoru. Na $30 \times 5 \times 30$ je vý-

sledný čas crawleru 2 462 ms oproti 873 ms ACO, přestože jedno provedení crawleru je řádově rychlejší než jedna iterace ACO.

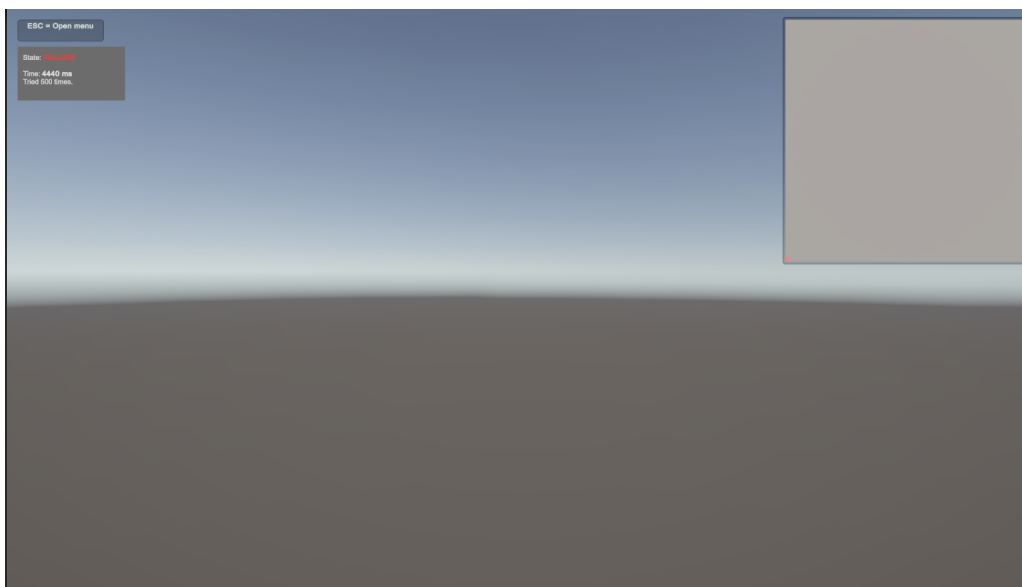


Obrázek 6.2: Výsledek GLS solveru na konfiguraci $30 \times 5 \times 30$. Restartů: 164, čas běhu: 1294 ms, délka cesty: 175. Crawler produkuje výrazně delší a méně optimální trasu než ACO solver. Zobrazený výsledek pochází z jednoho konkrétního běhu a nemusí odpovídat průměrným hodnotám uvedeným v tabulce 6.4

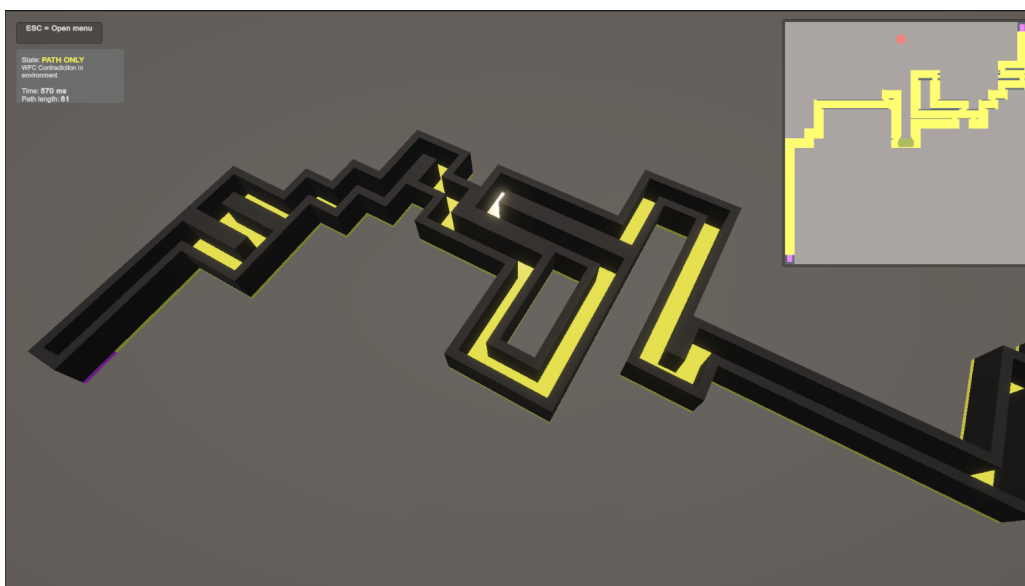


Obrázek 6.3: Výsledek ACO solveru na konfiguraci $30 \times 5 \times 30$ s $\beta = 7,9$. Čas běhu: 783 ms, délka cesty: 75. Trasa je optimálnější a generace rychlejší než u GLS.

Selhání hladového crawleru na plošných konfiguracích dokumentuje obrázek 6.4, který zachycuje stav mřížky po vyčerpání maximálního počtu pokusů. Obrázek 6.5 znázorňuje výstup v režimu izolované trasy (*Path Only Mode*) — aktivovaném tehdy, kdy crawler sestaví průchozí trasu, ale WFC fáze selže na kontradikci. V tomto režimu jsou vizualizovány výhradně zkolabované buňky páteřní trasy. Okolní buňky zůstávají ve stavu superpozice a nejsou instancovány. Srovnání obrázků 6.5 a 6.8 ilustruje kvalitativní rozdíl mezi trasami generovanými oběma přístupy: cesta crawleru sleduje lokálně výhodné kroky a vykazuje zbytečné zalomení, zatímco ACO trasa konverguje k cíli s nižším počtem odboček.



Obrázek 6.4: Selhání hladového crawleru na plošné konfiguraci — crawler uvázl v lokálním minimu a vyčerpал maximální povolený počet pokusů.



Obrázek 6.5: Režim izolované trasy: vizualizace páteřní cesty sestavené crawlerem v případě, kdy WFC fáze skončila kontradikčním stavem. Slouží jako záložní výstup pro jednopatrové konfigurace a umožňuje přímé srovnání délky a tvaru cesty s výstupem ACO solveru.

6.5 Finální verifikace ACO se sqrt-based škálováním

Po zafixování parametrů $\beta = 7,9$ a $\rho = 0,15$ byla provedena finální verifikace ACO solveru se sqrt-based dynamickým škálováním velikosti kolonie a počtu iterací na 100 nezávislých bězích (tabulka 6.5).

Dynamické škálování je definováno jako:

$$\text{colonySize} = \text{clamp}\left(\left\lfloor \sqrt{N} \cdot 0,5 \right\rfloor, 10, 300\right) \quad (6.1)$$

$$\text{maxIter} = \text{clamp}\left(\left\lfloor \sqrt{N} \cdot 0,2 \right\rfloor, 8, 100\right) \quad (6.2)$$

kde N označuje celkový počet buněk mřížky.

Konfigurace	Buněk	Colony	Iter	Čas (ms)	Délka cesty	Úspěšnost (%)
$10 \times 3 \times 10$	300	10	8	41	21,4	98
$15 \times 4 \times 15$	675	15	8	106	33,2	99
$20 \times 4 \times 20$	1 600	20	8	199	44,9	99
$25 \times 5 \times 25$	2 500	28	11	472	57,9	95
$30 \times 5 \times 30$	4 500	34	13	756	72,5	87

Tabulka 6.5: Finální verifikace ACO — sqrt-based škálování, $\beta = 7,9$, $\rho = 0,15$, 100 běhů

Pro mřížky do 1 600 buněk dosahuje solver 98–99% úspěšnosti s průměrnou dobou generace pod 200 ms. U konfigurace $30 \times 5 \times 30$ (4 500 buněk) klesá úspěšnost na 87 %, přičemž průměrná doba generace je 756 ms. Tato hodnota je přijatelná pro jednorázové použití při načítání úrovně – generace proběhne v době pod jednou sekundou.

Délka průměrné trasy roste přibližně lineárně s délkou hrany mřížky a odpovídá Manhattanské vzdálenosti mezi startovní buňkou $[0, 0, 0]$ a cílovou buňkou v protějším nejvyšším rohu mřížky. Pro konfiguraci $10 \times 3 \times 10$ činí tato vzdálenost $9 + 2 + 9 = 20$ buněk při naměřené průměrné délce 21,4 buněk, pro konfiguraci $30 \times 5 \times 30$ pak $29 + 4 + 29 = 62$ buněk při naměřené průměrné délce 72,5 buněk. Mírné překročení teoretického minima odpovídá topologickým překážkám prostředí.

6.6 Chování na řádově větších mřížkách

Zátěžový test sledoval chování ACO solveru na konfiguracích překračujících rozsah, pro který bylo sqrt-based škálování optimalizováno. Experiment probíhal ve dvou fázích: nejprve s dynamicky odvozenými parametry kolonie, poté s parametry nastavenými ručně.

6.6.1 Sqrt-based škálování na velkých konfiguracích

Při aplikaci dynamického škálování na mřížky s více než 12 500 buňkami klesá úspěšnost strmě až na nulovou hodnotu (tabulka 6.6). Na konfiguracích $75 \times 5 \times 75$ a $100 \times 5 \times 100$ kolonie v rámci přidělených iterací průchozí trasu vůbec nenalezne — hodnota průměrné délky cesty –1 indikuje, že žádný z pokusů neskončil úspěchem. Přestože je počet pokusů pro tyto konfigurace omezen na 3, dosahuje celkový průměrný čas generace 21–28 sekund, protože každý neúspěšný pokus vyčerpá celou alokaci iterací.

Konfigurace	Buněk	Čas (ms)	Pokusy	Délka cesty	Úspěšnost (%)
$50 \times 5 \times 50$	12 500	10 559	2,75	170,0	20
$60 \times 6 \times 60$	21 600	16 988	2,90	239,0	10
$75 \times 5 \times 75$	28 125	20 980	3,00	—	0
$100 \times 5 \times 100$	50 000	28 036	3,00	—	0

Tabulka 6.6: Zátěžový test ACO se sqrt-based škálováním — 20 běhů

Příčinou je, že sqrt-based vzorec přiděluje kolonii pro $N = 12\,500$ buněk velikost ≈ 56 agentů a ≈ 22 iterací. Tento počet je dostatečný pro prohledání lokálního okolí startovní buňky, avšak nepostačuje k tomu, aby feromonová stopa dostatečně zesílila na trase vedoucí k vzdálenému cíli — kolonie konverguje dříve, než vznikne globální feromonový gradient.

6.6.2 Manuální parametry

Aby bylo možné kvantifikovat chování systému za hranicí sqrt-based škálování, byly parametry kolonie pro konfigurace do $75 \times 5 \times 75$ nastaveny ručně nad hodnoty, které by dynamické škálování přidělilo. Výsledky jsou prezentovány v tabulce 6.7.

Konfigurace	Buněk	Colony	Iter	Čas (ms)	Délka cesty	Úspěšnost (%)
$50 \times 5 \times 50$	12 500	80	30	27 253	185,0	60
$60 \times 6 \times 60$	21 600	100	35	48 072	232,3	30
$75 \times 5 \times 75$	28 125	120	40	122 330	309,0	20

Tabulka 6.7: Zátěžový test ACO s manuálně nastavenými parametry — 20 běhů

Navýšení parametrů přináší měřitelné zlepšení — konfigurace $50 \times 5 \times 50$ dosahuje 60 % úspěšnosti oproti 20 % při dynamickém škálování — avšak úspěšnost s rostoucí mřížkou rychle klesá a na $75 \times 5 \times 75$ zůstává na 20 % i přes 3násobně větší kolonii, než by přidělilo sqrt-based škálování. Průměrný čas generace pro tuto konfiguraci přesahuje dvě minuty, což je pro interaktivní použití nepřijatelné.

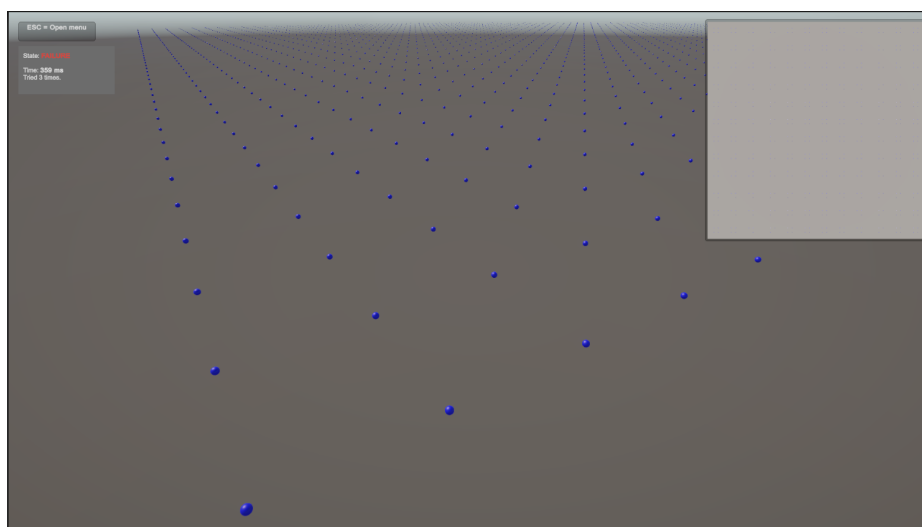
Tento výsledek naznačuje, že problém není primárně v podhodnocení parametrů škálovacím vzorcem. Se vzrůstajícím celkovým počtem buněk N roste počet stavů relevantních pro konstrukci trasy rychleji, než lineárně škálovaná kolonie dokáže pokrýt feromonovou stopou – přesná závislost nebyla v rámci této práce analyticky stanovena a představuje námět pro další výzkum.

Selhání zaznamenaná v obou fázích experimentu lze hypoteticky přičíst dvěma odlišným zdrojům: buď kolonie nenalezne buildable trasu v rámci přidělených iterací, nebo trasa nalezena je, ale WFC fáze skončí kontradikčním stavem při kolapsu zbývajících buněk. Přesné určení podílu každého z těchto zdrojů na celkové míře

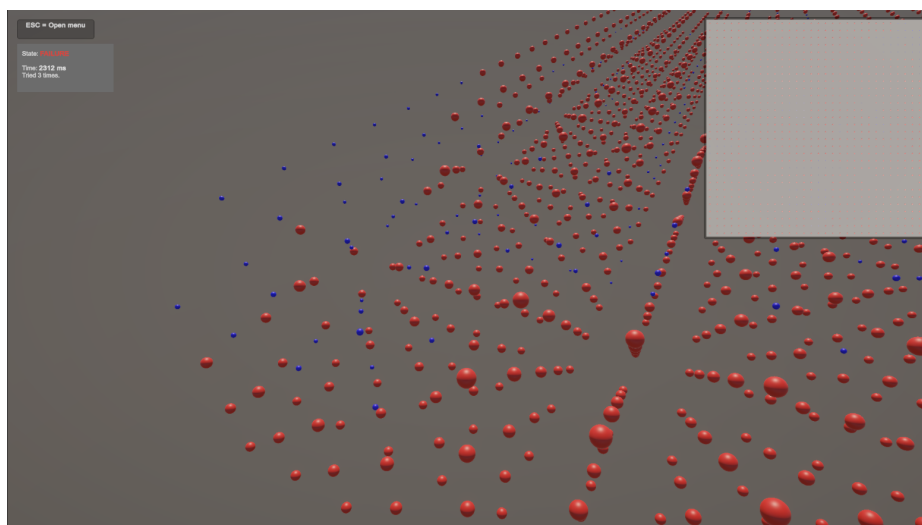
selhání nebylo v rámci této práce analyzováno. V budoucím výzkumu by proto bylo přínosné zaměřit se na prozkoumání této problematiky.

6.7 Vizuální analýza vlivu parametru beta

Vliv parametru β na chování kolonie je dobře patrný při srovnání feromonových heatmap generovaných po selhání ACO solveru. Obrázky 6.6 a 6.7 zachycují distribuci feromonů pro $\beta = 2,5$ na dvourozměrné a trojrozměrné konfiguraci. Feromony jsou rovnoměrně rozptýleny po celé mřížce, což svědčí o difúzní exploraci bez cílené konvergence. Oproti tomu obrázky 6.8 a 6.3 znázorňují výsledné prostředí pro $\beta = 7,9$, kde je trasa zřetelně soustředěna a okolní prostředí je plně zkolabováno.

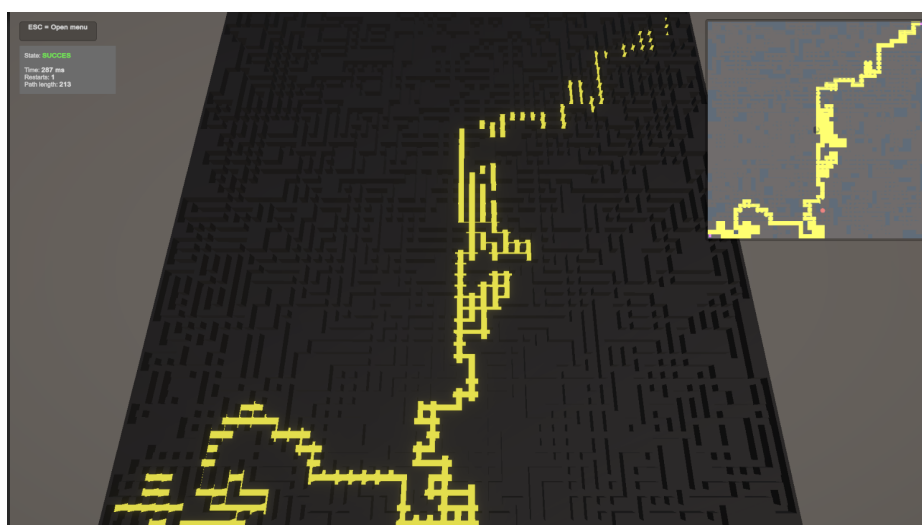


Obrázek 6.6: Feromonová heatmapa při $\beta = 2,5$ na dvourozměrné mřížce. Velmi slabá distribuce feromonů indikuje absenci konvergence kolonie k cíli.



Obrázek 6.7: Feromonová heatmapa při $\beta = 2,5$ na trojrozměrné mřížce $30 \times 5 \times 30$

Vysoká hustota feromonů (červené uzly) pokrývá většinu prostoru, zatímco oblast cílové buňky vykazuje minimální aktivitu (modré uzly). Nízká hodnota β potlačuje heuristickou složku přechodového pravidla, čímž kolonie stagnuje v již prozkoumané části prostoru namísto konvergence k cíli.



Obrázek 6.8: Úspěšná generace ACO solveru při $\beta = 7,9$ na dvourozměrné mřížce.



Obrázek 6.9: Příklad, kdy kolonie zvolila cestu přes nižší patro namísto kratší horizontální trasy— interakce feromonu a heuristiky.

Obrázek 6.9 ilustruje případ, kdy kolonie upřednostnila cestu vedoucí o patro níže namísto přímého horizontálního pohybu. Toto chování odpovídá interakci mezi feromonovou stopou a Manhattanskou heuristikou — v daném bodě mřížky byl součet $\tau(j)^\alpha \cdot \eta(j)^\beta$ pro buňku v nižším patře vyšší než pro horizontálního souseda, přestože horizontální cesta byla topologicky kratší. Jedná se o charakteristický projev heuristicky neinformovaného průzkumu svislé dimenze.

6.8 Diskuze a limitace

6.8.1 Odchylka od zadání: hard constraint namísto soft constraint

Jak je popsáno v sekci 1.3, implementace využívá pevná omezení namísto měkkých omezení specifikovaných v zadání. Důsledkem této volby je, že vliv ACO na výsledné prostředí je silnější než v soft constraint variantě: namísto statistického ovlivňování lokálních voleb ACO deterministicky určuje globální topologii průchozí osy. Teoretický předpoklad o riziku soft constraint integrace nebyl v rámci této práce experimentálně ověřen a jeho validace by vyžadovala samostatnou experimentální studii.

6.8.2 Reprodukovatelnost výsledků

Generátor pseudonáhodných čísel je parametrizován volitelným seedem předávaný rozhraním uživatelského vstupu. Nemá-li seed zadán, je vygenerován náhodně a jeho hodnota je zobrazena v rozhraní výsledku, což umožňuje přesnou reprodukci

konkrétního běhu. Benchmarky prezentované v této kapitole byly pořízeny bez fixace seedu a slouží jako průměrné chování přes různé běhy algoritmu.

6.8.3 Přenositelnost optimálního beta

Parametr $\beta = 7,9$ byl optimalizován výhradně pro mřížku $30 \times 5 \times 30$ s aktuální sadou modulů a socketů. Změna topologie prostoru (například dominance vertikálních přechodů), rozšíření sady modulů nebo výrazně odlišný poměr rozměrů mřížky může posunout optimální hodnotu β . Systém nenabízí mechanismus pro automatickou adaptaci tohoto parametru v závislosti na konfiguraci — β je hardcoded konstanta.

6.8.4 Absence automatizované verifikace výstupu

Korektnost výstupu je zajištěna návrhem jednotlivých komponent: ACO solver vrací trasu pouze tehdy, je-li každý její krok socketově kompatibilní, a WFC propagace udržuje konzistenci sousedství při každém kolapsu. Systém však neobsahuje samostatnou verifikační vrstvu, která by tyto invarianty ověřovala nezávisle nad celým výstupem po dokončení generace. Případná chyba na rozhraní mezi ACO a WFC fází by se projevila jako selhání generace, nikoli jako tichá produkce nekorektního výstupu.

6.8.5 Škálovatelnost nad 4 500 buněk

Zátěžové testy jednoznačně ukazují, že sqrt-based dynamické škálování parametrů kolonie přestává být dostatečné pro mřížky překračující přibližně 5 000–6 000 buněk. V tomto rozsahu by bylo vhodné buďto přejít na agresivnější škálovací strategii, nebo aplikovat prostorovou dekompozici — například generovat prostředí po úrovních (floor-by-floor) namísto jednoho globálního průchodu ACO.

Cílem práce byl návrh a implementace hybridního generátoru prostředí, který eliminuje strukturální slabinu algoritmu Wave Function Collapse spočívající v absenci záruky globální průchozosti. Navržené řešení tuto slabinu řeší předpočítáním páteřní trasy jako pevného omezení před spuštěním WFC fáze.

Implementovaný systém zahrnuje tři solvery nad společnou trojrozměrnou voxelovou mřížkou. Základní WFC Solver s výběrem buňky pomocí minimální haldy prokázal měřitelné výkonnostní zlepšení oproti lineárnímu prohledávání: na mřížce 4 500 buněk klesla doba strávená výběrem entropie z 81,59 ms na 1,99 ms, přičemž celková doba generace se snížila o přibližně 21 %. GLSHybridSolver implementující horolezecký algoritmus bez mechanismu návratu prokázal funkčnost na malých a středních konfiguracích, avšak na plošné mřížce $50 \times 1 \times 50$ vyčerpal veškeré dostupné pokusy (500) bez jediného úspěchu (viz sekce 6.4).

ACOHybridSolver dosahuje lepších výsledků: na téže konfiguraci $50 \times 1 \times 50$ dosáhl 100 % úspěšnosti s průměrnou dobou generace 185 ms. Na trojrozměrných konfiguracích ACO solver konsistentně generuje o 38–45 % kratší páteřní trasy než hladový crawler, při vyšší spolehlivosti a srovnatelném či nižším celkovém čase generace včetně opakování pokusů. Finální verifikace s dynamickým škálováním parametrů kolonie na základě $\sqrt{N}$ na 100 nezávislých bězích potvrdila 98–99 % úspěšnost pro mřížky do 1 600 buněk a 87 % pro 4 500 buněk. Experimentálně optimalizovaný parametr $\beta = 7,9$ přechodového pravidla ACO byl ověřen třífázovým sweepem jako stabilní volba pro prezentované trojrozměrné konfigurace.

Práce zároveň nechává několik otevřených oblastí. Dosažená implementace využívá pevná omezení (*hard constraints*) namísto původně uvažovaných měkkých omezení (*soft constraints*), kdy by feromonové hodnoty přímo ovlivňovaly výběr variant uvnitř WFC smyčky – tato architektura je hlavním směrem dalšího vývoje. Zátěžové testy ukázaly, že škálování parametrů kolonie přes $\sqrt{N}$ přestává být dostatečné pro mřížky překračující přibližně 5 000–6 000 buněk. Potenciálním směrem je prostorová dekompozice do vrstev nebo agresivnější škálovací strategie. Přenositelnost hodnoty $\beta = 7,9$ na konfigurace výrazně odlišných rozměrů nebo topologických charakteristik nebyla systematicky ověřena. Výsledky benchmarků nejsou

deterministicky reprodukovatelné, neboť měření byla pořízena bez fixace seedu. Implementovaný mechanismus volitelného seedování tuto reprodukovatelnost pro budoucí experimenty umožňuje. Dále také nebylo analyticky rozlišeno, zda selhání na velkých konfiguracích pramenů z neschopnosti kolonie nalézt buildable trasu v rámci alokovaných iterací, nebo z následné kontradikce při WFC fázi – kvantifikace těchto dvou zdrojů selhání by poskytla přesnější podklady pro optimalizaci systému. Přenositelnost algoritmu GLSHybridSolver mimo původní 3D kontext byla ověřena jeho nasazením v externím projektu předmětu KIV/PH, kde sloužil jako generátor závodní tratě na dvojrozměrné mřížce s dynamicky definovaným cílem a checkpointy — viz sekce 4.2.1.

Uživatelská příručka



Tato příručka popisuje základní ovládání aplikace a její nástroje.

A.1 Spuštění a hlavní menu

Po spuštění aplikace se zobrazí hlavní menu se třemi nastavitelnými parametry. První parametr určuje *velikost mřížky* (Grid size). Dostupné předvolby jsou uvedeny v tabulce A.1. Druhý parametr volí *algoritmus* použitý pro generaci prostředí a třetí parametr určuje seed pro generaci.

Označení	Rozměry (X × Y × Z)	Buněk	Typ
Small 2D	10 × 1 × 10	100	2D
Medium 2D	25 × 1 × 25	625	2D
Large 2D	50 × 1 × 50	2 500	2D
Small 3D	10 × 3 × 10	300	3D
Medium 3D	20 × 4 × 20	1 600	3D
Large 3D	30 × 5 × 30	4 500	3D

Tabulka A.1: Dostupné předvolby velikosti mřížky

Pure WFC implementuje čistý algoritmus Wave Function Collapse bez předpočítané trasy — je rychlý, avšak na větších konfiguracích náchylný ke kontradikci a nezaručuje průchoznost prostředí.

GLS Crawler nejprve předpočítá páteřní trasu pomocí horolezeckého algoritmu a poté WFC zaplní okolní prostředí; je vhodný pro malé a střední konfigurace.

ACO Hybrid využívá optimalizaci mravenčí kolonie pro předpočítání páteřní trasy a je doporučenou volbou pro střední a velké konfigurace.

Pro konfiguraci Large 2D (50 × 1 × 50) je doporučeno používat výhradně ACO Hybrid. Systém na nevhodnou kombinaci solveru a konfigurace upozorní varovnou hláškou v menu ještě před spuštěním generace.

A.2 Generování prostředí

Po výběru parametrů uživatel stiskne tlačítko `GENERATE`. Volitelné textové pole pro seed umožňuje zadat celé číslo pro deterministickou reprodukci konkrétního výsledku. Je-li prázdné, je seed vygenerován náhodně a jeho hodnota je zobrazena v rozhraní výsledku. Systém inicializuje mřížku, spustí zvolený solver a v případě detekce kontradikce automaticky opakuje generaci. Maximální počet pokusů je škálován dynamicky podle velikosti mřížky; pro ACO Hybrid jsou přiděleny nejvýše tři pokusy. Průběh generace a výsledek (*SUCCESS* nebo *FAILED*) jsou průběžně zobrazovány v levém horním rohu obrazovky spolu s počtem dosud provedených pokusů.

A.3 Pohyb ve vygenerovaném prostředí

Po úspěšné generaci je uživateli zpřístupněna kamera bez kolizních omezení. Ovládání je shrnuto v tabulce A.2.

Vstup	Funkce
W / S	Pohyb vpřed / vzad
A / D	Pohyb vlevo / vpravo
Pohyb myši	Otočení pohledu
E	Pohyb nahoru
Q	Pohyb dolů
Shift	Sprint
ESC	Návrat do hlavního menu

Tabulka A.2: Klávesové zkratky a ovládání kamery

A.4 Minimapa a diagnostické nástroje

V pravém horním rohu obrazovky je trvale zobrazena 2D minimapa aktuálního patra. Zobrazené patro se automaticky přepíná podle vertikální polohy kamery; poloha kamery je na minimapě vyznačena ikonou kolečka.

Pokud ACO Hybrid selže ve všech pokusech, systém automaticky zobrazí *feromonovou heatmapu*: uzly mřížky jsou vykresleny jako prostorové objekty, jejichž velikost a barevné kódování (od modré pro nízkou koncentraci po červenou pro vysokou) zobrazí výslednou distribuci feromonů. Tato vizualizace umožňuje identifikovat oblasti, v nichž agenti opakovaně selhávali při hledání průchozí trasy.

Pokud GLS Crawler sestaví páteřní trasu, ale následný WFC průchod skončí kontradikčním stavem, systém přejde do *režimu izolované trasy (Path Only Mode)*:

zobrazeny jsou výhradně buňky tvořící páteřní trasu, okolní prostředí není instancováno. Tento režim je dostupný pouze pro jednopodlažní konfigurace.

Bibliografie

- [Cor+22] CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. *Introduction to Algorithms*. 4. vyd. Cambridge, MA: MIT Press, 2022. ISBN 9780262046305.
- [DG97] DORIGO, Marco; GAMBARDELLA, Luca Maria. Ant colony system: a cooperative learning approach to the traveling salesman problem. *IEEE Transactions on Evolutionary Computation*. 1997, roč. 1, č. 1, s. 53–66.
- [DS04] DORIGO, Marco; STÜTZLE, Thomas. *Ant Colony Optimization*. Cambridge, MA: MIT Press, 2004.
- [Gum16] GUMIN, Maxim. *Wave Function Collapse Algorithm* [online]. GitHub, 2016 [cit. 2026-04-28]. Dostupné z: <https://github.com/mxgmn/WaveFunctionCollapse>.
- [KS17] KARTH, Isaac; SMITH, Adam M. Wave Function Collapse is constraint solving in the wild. In: *Proceedings of the 12th International Conference on the Foundations of Digital Games*. ACM, 2017, 68:1–68:10.
- [Kle19] KLEINEBERG, Marian. *Infinite procedurally generated city with the Wave Function Collapse algorithm* [online]. 2019. [cit. 2026-04-28]. Dostupné z: <https://marian42.de/article/wfc/>.
- [Liu+26] LIU, Jianjuan et al. Application of ant colony optimization with wide-area search concept in path planning for complex large-scale environments. *Engineering Applications of Artificial Intelligence*. 2026, roč. 165, s. 113523. ISSN 0952-1976. Dostupné z DOI: <https://doi.org/10.1016/j.engappai.2025.113523>.
- [RN21] RUSSELL, Stuart J.; NORVIG, Peter. *Artificial Intelligence: A Modern Approach*. 4. vyd. Hoboken, NJ: Pearson, 2021. ISBN 9780134610993.
- [STN16] SHAKER, Noor; TOGELIUS, Julian; NELSON, Mark J. *Procedural Content Generation in Games*. Springer, 2016. ISBN 9783319427164.

- [Zap23] ZAPPE, Dominik. *Tvorba a řešení bludišť*. Plzeň, 2023. Bakalářská práce. Západočeská univerzita v Plzni, Fakulta aplikovaných věd, Katedra informatiky a výpočetní techniky. Vedoucí práce Prof. Dr. Ing. Ivana KOLINGEROVÁ.

1101001
101011000011100010 1100001
101011010101 1100001

11010011101101001
01100001 10101
11000101011 01